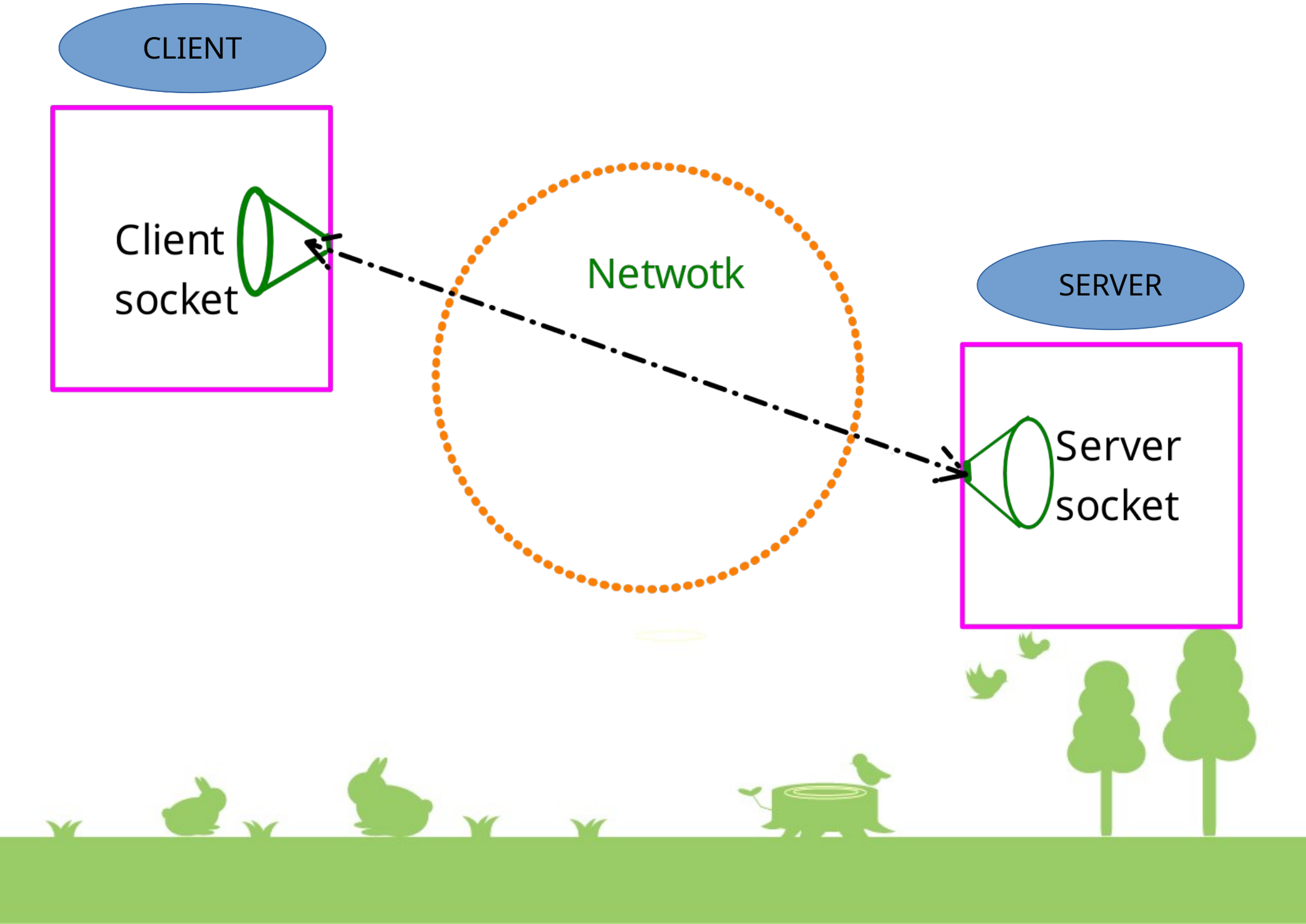
SOCKET PROGRAMMING

Mr.GOPI

What is socket programming

Socket programming is a way of connecting two nodes on a network to communicate with each other.

It involves the use of **sockets**, which are communication endpoints that allow data to be sent and received between two nodes.


CLIENT
Client
socket
Netwotk
SERVER
Server
socket

Socket API

We will use **socket API** with C programming in linux environment

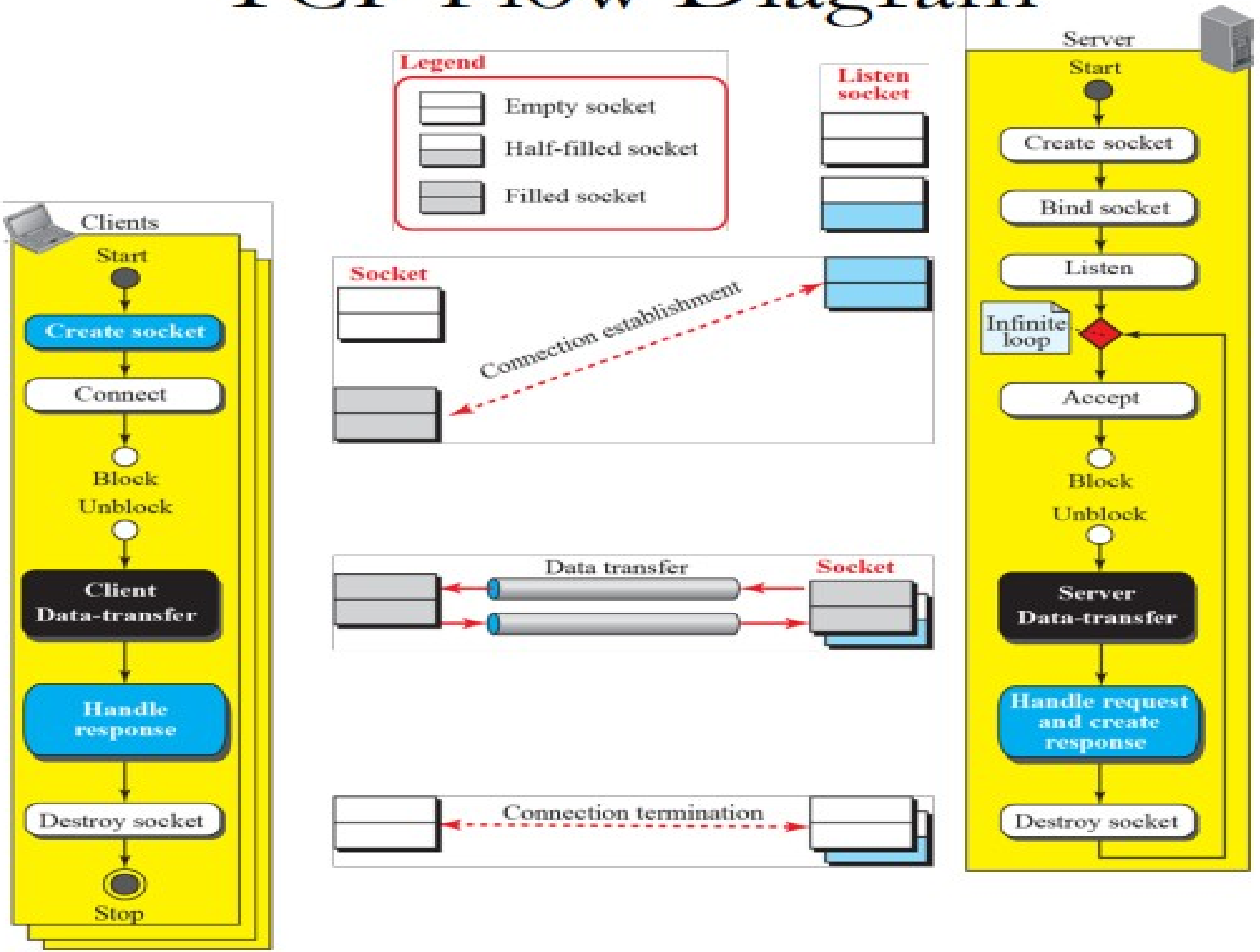
Socket programming contains many **data structures** , **functions** and **system calls**.

Essential types of sockets

SOCK_STREAM :-

- TCP
- reliable delivery
- in-order guaranteed
- connection-oriented
- bidirectional

TCP Flow Diagram



SOCK_STREAM (TCP)

In client and server model, few system calls require.

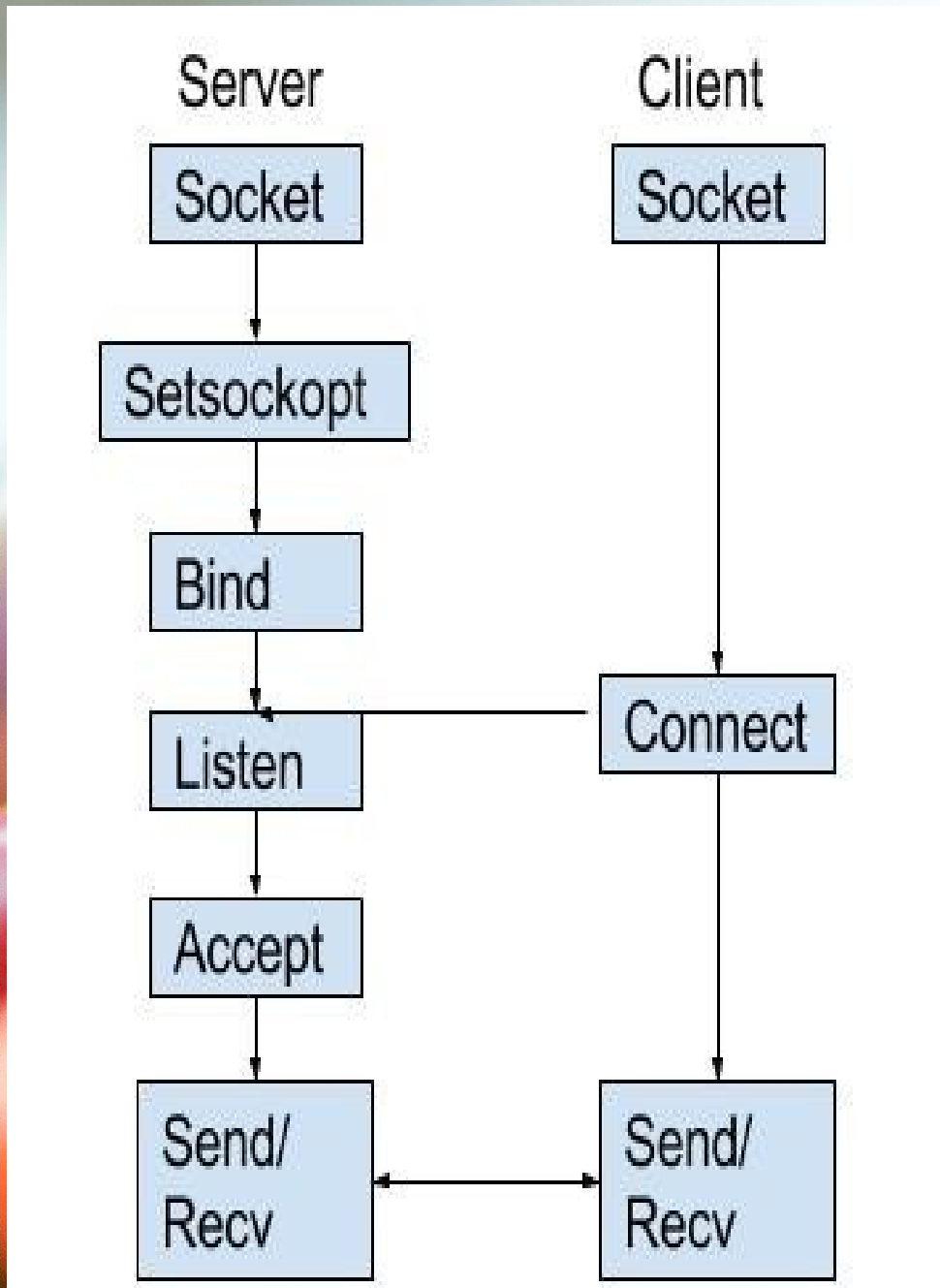
Server side:

socket , bind , listen , accept

Send / recv

Client side:

Socket , connect , send / recv

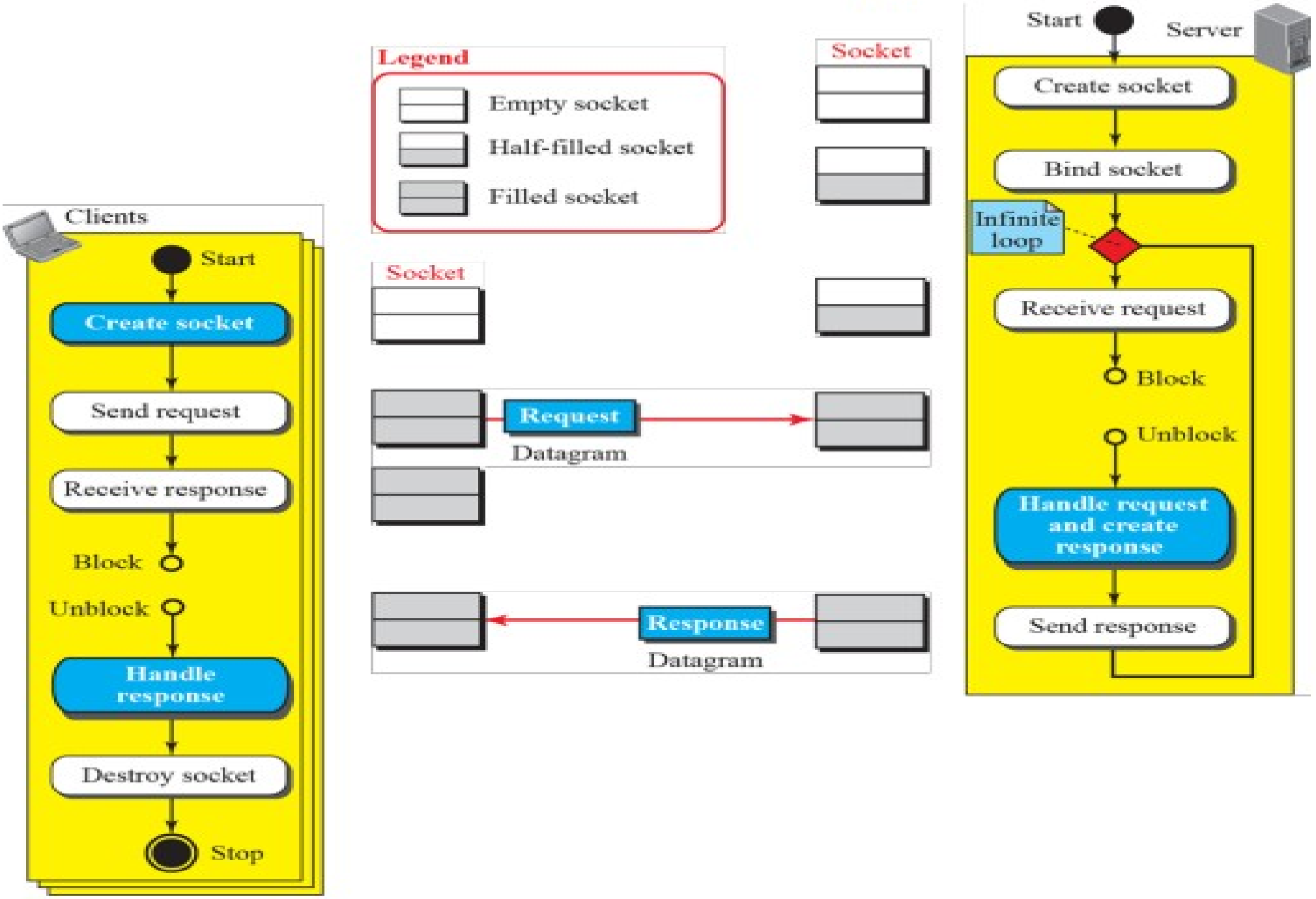


Essential types of sockets

SOCK_DGRAM :-

- UDP
- un-reliable delivery
- no order guarantee
- connectionless
- can send and receive

UDP Flow Diagram



SOCK_DGRAM (UDP)

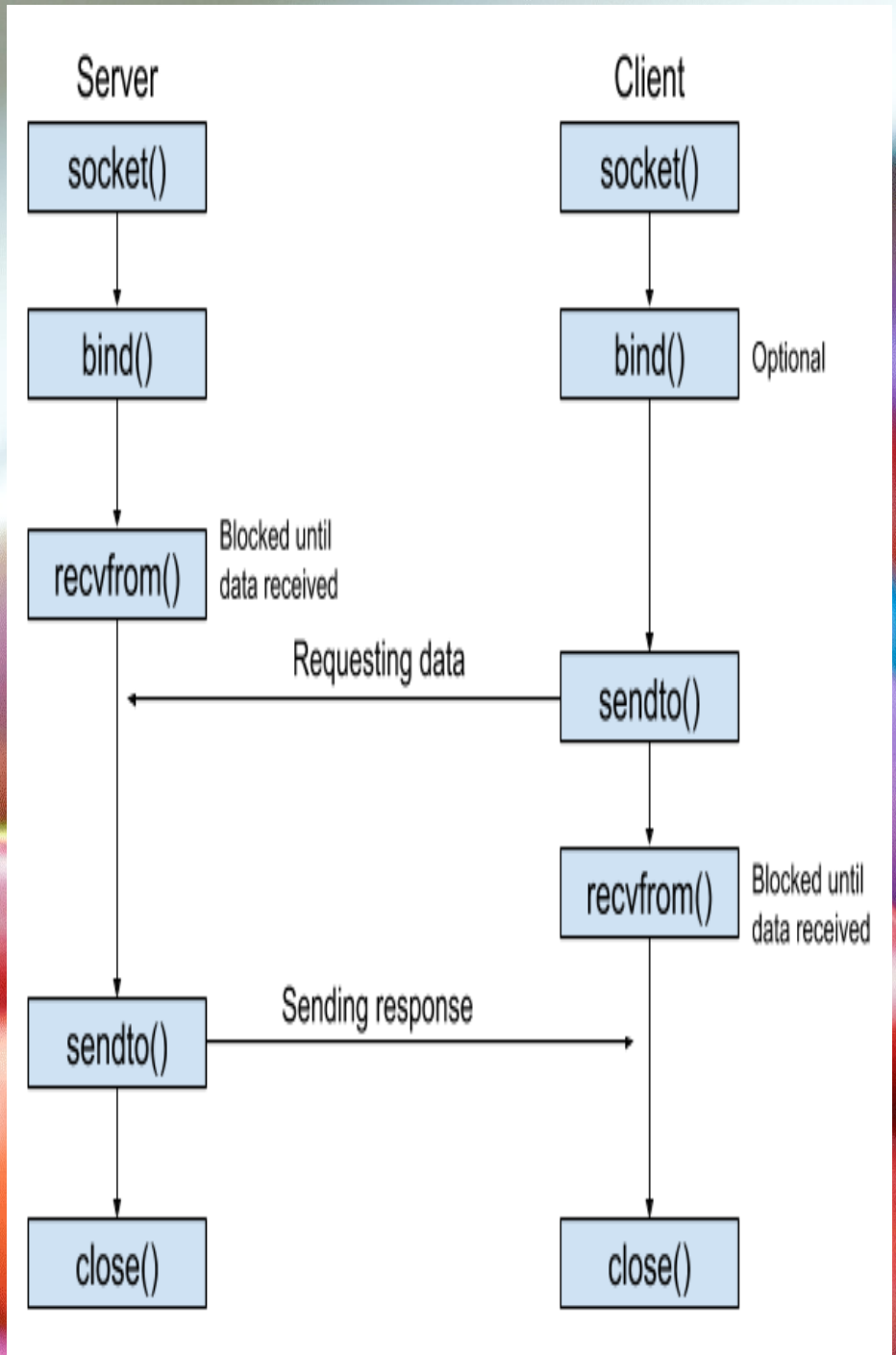
In client and server model, few system calls require.

Server side:

socket , bind , sendto / recvfrom

Client side:

Socket , bind , sendto / recvfrom



System call: socket()

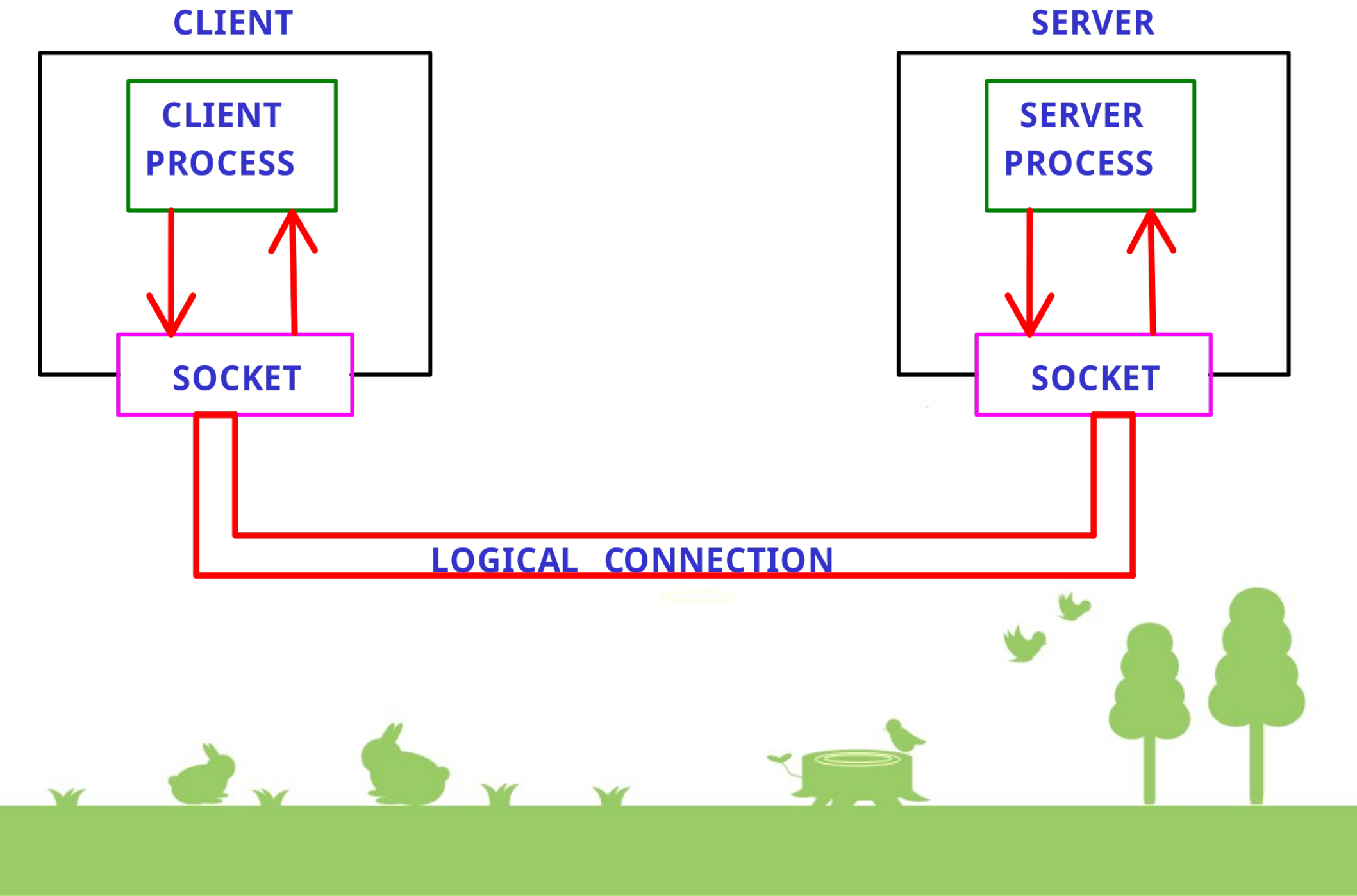
- Creating a socket

```
int socket( int family , int type , int protocol );
```

- This call is used in both server and client

- Sockets allow communication between two different processes on the same or different machines.

- To be more precise, it's a way to talk to other computers using file descriptors.


CLIENT
SERVER
CLIENT PROCESS
SERVER PROCESS
SOCKET
SOCKET
LOGICAL CONNECTION

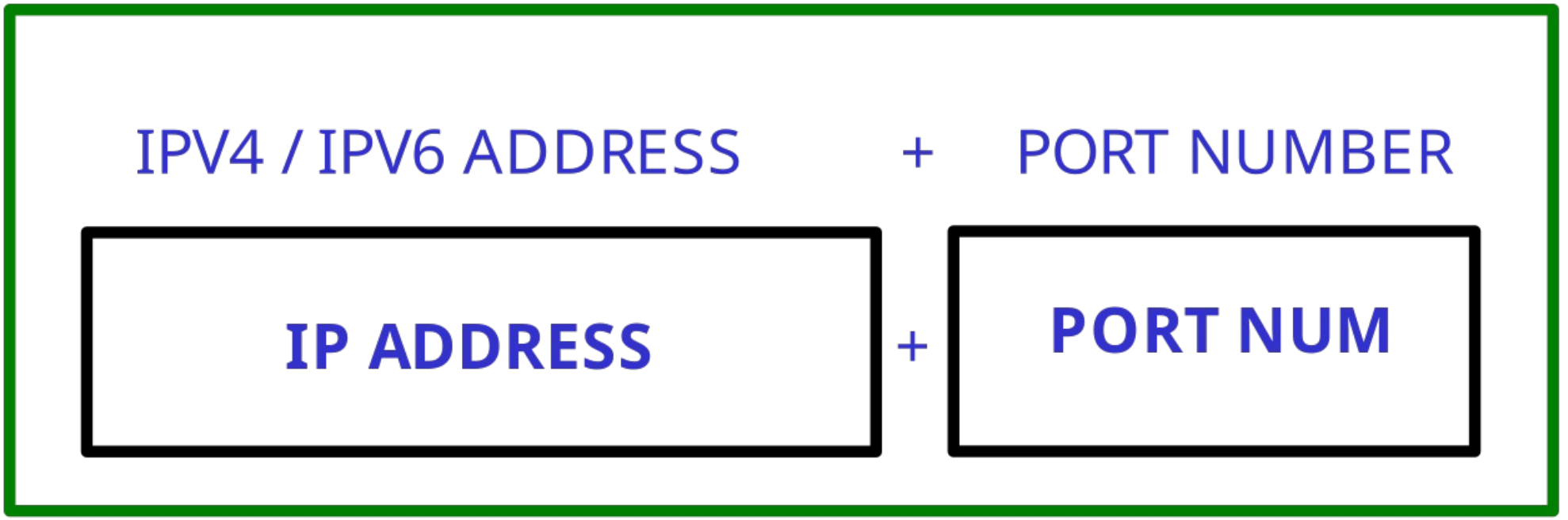
System call: bind()

- It assigns a local protocol address to a socket.

```
int bind(int sockfd, const struct sockaddr *servaddr,
socklen_t addrlen);
```

- This call is used in server and in client it is optional

- With the Internet protocols, the address is the combination of an IPv4 or IPv6 address (32-bit or 128-bit) address along with a16 bit port number.



SOCKET ADDRESS

System call: listen()

- The listen() function converts an unconnected socket into a passive socket.

```
int listen(int sockfd, int backlog);
```

- This call is used in server and in client it is optional

- It creates connection request queue for incoming connection

System call: accept()

- The accept() call is used by a server to accept a connection request from a client.

  - int accept(int socket, struct sockaddr *address int *address_len);

- The call accepts the first connection on its queue of pending connections for the given socket .

- When a connection is available, the accept call creates new socket for use to send / read data from the process that requested the connection.

- If the queue has no pending connection requests, accept() call blocks the caller unless socket is in non-blocking mode.



System call: connect()

- The connect() call is used by a **client** to establish a connection between client and server.

  - int connect(int socket, struct sockaddr *address
    int address_len);

- The connect() call performs two tasks when called for a stream socket.

- First, it completes the binding necessary for a stream socket.
- Second, it attempts to make a connection to another socket.



System call: send() / recv()

- The send() , recv() calls used by **TCP client – server model** to send and recv MSG's.

  - ssize_t send (int sockfd, const void buf[.len], size_t len
    int flags);
  - ssize_t recv (int sockfd, void buf[.len], size_t len,
    int flags);

System call: sendto()

- The sendto() call used by **UDP client – server model** to send MSG's.

  - ssize_t **sendto** (int sockfd, const void buf[.len], size_t len, int flags, const struct sockaddr *dest_addr, socklen_t addrlen);

System call: recvfrom()

- The recvfrom() call used by **UDP client – server model** to recv MSG's.

  – ssize_t **recvfrom** (int sockfd, void buf[restrict .len], size_t len, int flags, struct sockaddr * src_addr socklen_t * addrlen);

System call: setsockopt()

- manipulate options for the socket referred to by the file descriptor sockfd.

  - int **setsockopt** (int sockfd, int level, int optname, const void optval[.optlen], socklen_t optlen);



```
Int main()
{
printf("THANK YOU");
}
```